

promptlab Hackathon — Instructor Pack

Do not distribute. Students receive the rubric weights and the hidden-suite categories. Nothing else here.

1. Why this task works

The module taught Python, prompt engineering, context engineering, Claude Code and spec-driven development. Four of those five are about *using AI well*, which is exactly what a conventional build task fails to measure.

promptlab measures it three ways that cannot be faked:

- **The prompt improvement track.** Students must improve a weak prompt and prove the gain with their own harness. The stub model genuinely rewards technique — measured differential is **0.63** → **0.97** accuracy between the weak starter prompt and a well-engineered one. A team that does not understand prompt engineering cannot move that number, and the number is in their own report.
- **The non-determinism requirement.** Ask any assistant to build a prompt test runner and it returns a clean pass/fail runner, because that is what test runners are. Flakiness is the actual problem, and the pattern-matched answer misses it. Teams discover this around hour 6 when they first raise the temperature.
- **Spec-first, verified by git.** Commit order is checked, not claimed.

Coverage check

Taught	Where it is exercised
Python	Subprocess handling, JSON, regex, unittest , CLI, stdlib-only
Prompt engineering	IMPROVEMENT.md is 15% and is scored on the measured gain plus the reasoning behind it
Context engineering	USAGE.md written for an agent consumer; CLAUDE.md graded
Claude Code	Required driver; history and context files inspected
Spec-driven development	Spec-first enforced; curveball re-tests it under pressure

2. The stub model — what students are not told

`stubmodel.py` is fully documented to students and they may read it. That is deliberate: this hackathon is a construction task, not a discovery task. What matters is that they cannot *bypass* it, which MUST 3 enforces by requiring subprocess invocation.

Prompt features it detects

Feature	Detected by	Effect
format	mentions JSON, or a fenced block	Emits JSON instead of prose
output_only	"only output", "no preamble", "nothing else", etc.	Drops preamble and tail

Feature	Detected by	Effect
examples	two or more Input: / Example: / User: line starts	+0.15 accuracy
labels	four or more category names present	+0.20 accuracy
edge_rule	"unclear", "if unsure", "ambiguous", etc.	+0.10 accuracy, and unlocks the unclear answer entirely
role	starts with "you are"	+0.03
length_limit	"under N words"	Truncates to N words and drops the verbose filler
policy	"redact", "sensitive", "never include", "pii"	Prevents refusal on sensitive tickets

Base accuracy is 0.50; ceiling is 0.98.

The **edge_rule** gate is the most instructive. Four tickets in the corpus have ground truth **unclear**. A prompt with no instruction about ambiguity can never answer them correctly, no matter what else it does. Teams that only add examples and a label list plateau and cannot work out why. This is the single best viva topic in the project.

Failure modes it simulates

Mode	Trigger	What it teaches
Preamble noise	no output_only	not_contains assertions, output hygiene
Prose instead of JSON	no format	Format instructions matter
Key reordering	temperature > 0, ~35%	String equality on JSON is a bug; structural assertions are not optional
Code fences	temperature > 0, ~25%	The fenced-JSON decision in their spec
Preamble slip	temperature > 0, ~15%	Even a correct instruction is not obeyed 100% of the time
Casing and whitespace jitter	temperature > 0	normalize and ignore_case exist for a reason
Truncation	--max-tokens low	finish: "length" , invalid JSON mid-object
Refusal	sensitive ticket, no policy in prompt	finish: "refusal" as a third outcome

Determinism

- temperature 0 → seeded from SHA-256 of prompt plus input. Byte-identical, always.
- temperature > 0 with **--seed** → reproducible via (**seed**, **call_index**, **prompt**, **input**).
- temperature > 0 without **--seed** → **os.urandom**. Genuinely random. **Verified: 12 calls produced 11 distinct outputs.**

Reference answer

instructor/classify_reference_v2.txt is a well-engineered prompt that scores 0.97. Do not distribute it. Use it to sanity-check the corpus before the event and to show in the debrief.

3. Rubric

Criterion	Weight	0–40%	41–70%	71–100%
Functionality	30	<code>run</code> incomplete, assertions missing	All eight assertions, pass/fail only, compare thin	Flaky classification correct, compare complete, survives the swapped model binary
Understanding (viva)	20	Cannot explain own code	Explains what, not why	Fluent on counterfactuals and extensions
Prompt improvement	15	No measurable gain, or gain unexplained	Gain shown, reasoning thin	Large measured gain, four-plus iterations, the failed change documented honestly
Curveball	15	Not attempted	Partial, hacked in	Absorbed cleanly; the assertion layer was already pluggable
Spec, tests, context	20	Spec late or generic, no tests	Spec first, tests thin	Spec-first with visible revision, real tests, <code>USAGE.md</code> an agent could act on

Hard rules, announced at kickoff:

- **Understanding gates Functionality.** Cannot explain a component → it scores zero.
- **Spec-first is binary.** Implementation in the first commit → Spec/tests/context = 0. Check `git log --reverse --stat`.
- **Importing `stubmodel` instead of subprocessing it caps Functionality at 40%.** It is MUST 3 and it is what makes the swapped-binary test meaningful.

4. The curveball — release at 14:00

Announce verbally and in writing simultaneously. No hints beforehand. No deadline extension.

> **SPEC CHANGE — effective immediately, all teams.** > > Assertions must now compose. Implement three new types: > > - `all_of` — field `assertions`, a list of assertion objects. Passes when every child passes. > - `any_of` — field `assertions`, a list of assertion objects. Passes when at least one child passes. > - `json_subset` — field `value`, an object. Passes when the output parses as JSON and every key/value in `value` appears in it, at any depth of nesting. > > `all_of` and `any_of` may nest inside each other, to any depth. Your per-assertion counts in the report must still work: a nested assertion reports as one top-level entry, and `failures` must identify which child failed. > > Update `SPEC.md` **before** you implement. The spec commit must precede the implementation commit, exactly as at the start of the event.

Why this one. The seam it stresses is whether assertion evaluation is a **pure function returning a result**, or a procedure that writes into the report as it goes. Teams with a registry and a return value add nesting in twenty minutes. Teams with a 200-line `if/elif` chain that appends to a report list — the exact shape Claude Code produces when asked to "implement these eight assertion types" in one prompt — find that nesting is impossible without unpicking the whole thing.

It also re-tests spec discipline under time pressure, which is when process actually reveals itself.

Backup curveballs:

- `promptlab gate --baseline --candidate --max-regressions N`, exit 1 when exceeded
- Suites may `include` shared case files by path
- `--format junit` as a second reporter

5. Hidden suites

Prepare these before the event and verify each against a reference implementation. Run in this order, in front of the team.

#	Suite / action	Pass condition
1	<code>promptlab doctor</code> on a clean clone	Runs, reports model reachable, no traceback
2	Smoke suite, temperature 0	Exit 0 or 2 as appropriate, report matches schema
3	Empty suite (<code>"cases": []</code>)	Exit 0, valid report, totals all zero
4	Case with <code>"assert": []</code>	Does not crash. Their spec should say whether a case with no assertions passes
5	Malformed: missing <code>prompt_file</code> ; <code>runs</code> as a string; unknown assertion type	Exit 1, readable message naming the problem
6	<code>prompt_file</code> points at a nonexistent path	Exit 1 or 4, one line, no traceback
7	Temperature 0.7, <code>--runs 10</code>, cases that drift	Flaky cases labelled <code>flaky</code>, not <code>pass</code>. The headline test.
8	Suite where every case fails	Exit 2
9	<code>matches</code> with invalid regex <code>"([unclosed"</code>	Exit 1, no traceback
10	<code>json_field_equals</code> with a deep dotted path	Correct resolution, no crash on a missing path
11	<code>--max-tokens 12</code> , output truncated mid-JSON	<code>finish_is: length</code> passes, <code>json_valid</code> fails, no crash
12	Sensitive ticket, prompt without a policy line	<code>finish_is: refusal</code> passes
13	Same suite run twice at temperature 0, reports diffed	Identical apart from timing fields
14	<code>compare</code> on two reports from different suites	Warns, does not silently produce nonsense
15	<code>compare</code> on a report against itself	Warns on matching <code>prompt_hash</code>
16	Swapped model binary — same CLI contract, different outputs	Harness still runs and reports correctly
17	Their <code>IMPROVEMENT.md</code> numbers, reproduced live	Claimed gain matches what you measure

Tests 7 and 16 are the discriminators. Test 7 catches the pass/fail assumption. Test 16 catches any team whose assertions were tuned to the specific strings this model emits rather than to the contract.

Build the swapped binary by editing `PREAMBLES`, `TAILS` and the prose format in a copy of `stubmodel.py`. Keep the CLI and the output JSON shape identical.

6. Viva bank

Four to six minutes. **Judges pick the person and the file.**

On the design

- Walk me through a single case from suite file to report entry.
- What is your flaky threshold and why that number?
- Is fenced JSON valid JSON in your implementation? Show me where that decision lives.
- Where does an assertion get evaluated? What does that function return?

On the prompt work

- Which change to the prompt produced the biggest gain? How do you know it was that one?
- Show me the change that did not help. Why did you expect it to?
- Four tickets have ground truth **unclear**. What does a prompt need in order to get those right? (*The edge-case rule. Teams that never cracked this will not know.*)
- Your v2 costs more tokens than v1. Was that worth it?

Counterfactual — weight heavily

- Delete this line. What breaks, and where does the error surface?
- I raise `--runs` from 3 to 300. What changes in your report, and what changes in your conclusions?
- The model starts returning an empty string. What does your harness do?
- Two cases have the same `id`. What happens?

On the assistant

- Show me `CLAUDE.md`. Why is that in there and not something else?
- Which prompt in `PROMPTS.md` did the most work? Remove that sentence — what changes?
- **Where did Claude Code get this confidently wrong?** (*The expected answer is the pass/fail assumption. A team with no answer here did not build this.*)

Closer

- Which part of this are you least confident in?

7. Checkpoints

Time	Mentors verify
2:00	<code>SPEC.md</code> committed, no code. Push hard on the flaky policy — "we'll use a threshold" is not a policy. Ask what happens at <code>--runs 2</code> .
6:00	One suite runs end to end. Ask whether they have tried temperature above zero yet. Do not tell them what happens.
12:00	<code>compare</code> works and <code>IMPROVEMENT.md</code> has started. Teams that left the prompt track until the end lose 15% cheaply — nudge them.
15:00	Post-curveball plan exists. Ask: what does your assertion evaluator return right now?
21:00	Fresh clone yourself: <code>doctor</code> , smoke suite, <code>python -m unittest</code> .

The 6:00 checkpoint is where the project turns. Teams that have only run at temperature 0 have not met the real problem yet, and the later they meet it the more they rewrite.

8. Signals of unowned work

- Report has `passed/failed` only — no flaky concept anywhere
- `import stubmodel` or a reimplement of its logic
- A single 200-line `if/elif` chain evaluating assertions, appending to a list as it goes
- Assertions tuned to literal strings this model emits ("Sure! Here's")
- `IMPROVEMENT.md` shows one iteration and a round number
- Tests that assert `True == True`, or no tests despite MUST 21
- `USAGE.md` is a command listing with no exit-code guidance
- Journal question 3 empty — nearly impossible for a team that genuinely built this
- One commit near the deadline
- Team cannot reproduce their own `IMPROVEMENT.md` numbers live

Handle it in the viva, never by accusation. Ask the counterfactuals and let the score speak.

9. Judging budget

```
per team = doctor + demo 3 + hidden suites 5 + viva 6 + gap 2 = 16 min
16 teams × 16 = 4h 16m
```

Over the window? Script suites 1–8, run them on all teams first, full judging for the top N. **Do not cut the viva.**

10. Prep checklist

- [] Verify the stub model on a clean machine; confirm the 0.63 → 0.97 differential
- [] Build the swapped model binary for test 16
- [] Build all 17 hidden suites and verify against a reference implementation
- [] Write the scripted runner for suites 1–8
- [] Package starter materials: `stubmodel.py`, `prompts/classify_v1.txt`, `data/tickets.json`, `data/ticket_sample.txt`, `suites/smoke.json`, templates
- [] **Keep `instructor/classify_reference_v2.txt` out of the student bundle**
- [] Confirm Python 3.10+ and working Claude Code on every machine the day before
- [] Brief judges on the three hard rules in §3
- [] Assign someone to post the curveball in writing at exactly 14:00

11. Debrief — 30 minutes

1. **Reveal the model's feature table.** Show which prompt signals moved the number. Ask teams to guess before you show it.
2. **Show the reference prompt** and its 0.97. Compare with the room's spread.
3. **Ask who got the `unclear` tickets right, and what they added to the prompt.** Most teams never work out that it needs an explicit ambiguity rule.
4. **Ask when each team first ran at temperature above zero.** Plot it against final rank in front of them. The correlation is stark and the point makes itself.
5. **Close with the question the hackathon exists for:** *"What did you believe about your prompt before you measured it?"*

That discussion, with the whole cohort in the room, is the most valuable twenty minutes of Module 1.